Aula 08 — Repetições aplicadas

Contadores, acumuladores, validação de dados e laços aninhados

Dieimes Nunes

Retomando a Aula 07

Na aula anterior, trabalhamos com repetição usando `for` e `range()`.

Vimos que o `for` é muito útil quando sabemos a quantidade de repetições.

Exemplos:

```
for numero in range(1, 6):
    print(numero)
```

```
for contador in range(10, 0, -1):
    print(contador)
```

Hoje vamos aplicar repetições em problemas mais completos.

O que vamos aprofundar hoje

Nesta aula, vamos trabalhar com:

- contadores;
- acumuladores;
- validação de dados;
- repetição com condição;
- laços aninhados;
- pequenos relatórios com dados digitados pelo usuário.

A ideia é sair de repetições simples e começar a resolver problemas mais próximos de situações reais.

Por que repetir não é apenas contar?

No começo, usamos repetição para contar:

```
1, 2, 3, 4, 5...
```

Mas em programas reais, repetimos para:

- cadastrar várias pessoas;
- somar vários valores;
- contar quantas respostas foram dadas;
- validar informações;
- repetir um menu até o usuário escolher sair;
- analisar dados e gerar um resumo.

Situação inicial

Imagine que uma escola precise registrar as notas de uma turma.

O programa deve:

1. perguntar quantos alunos serão cadastrados;
2. receber uma nota para cada aluno;
3. somar as notas;
4. calcular a média da turma;
5. mostrar o resultado final.

Antes de programar, precisamos pensar:

Atividade 1 — Pensando no problema

No caderno, responda:

Problema: calcular a média de uma turma.

1. Qual informação o usuário precisa informar primeiro?
2. Qual informação será repetida várias vezes?
3. O que precisa ser somado?
4. O que precisa ser contado?
5. Qual será a saída final?

Depois, compare sua resposta com um colega.

Contador

Um **contador** é uma variável usada para contar quantas vezes algo aconteceu.

Exemplo:

```
contador = 0

contador = contador + 1
contador = contador + 1
contador = contador + 1

print(contador)
```

Resultado:

```
3
```

O contador começou em zero e foi aumentando de um em um.

Onde usamos contadores?

Contadores aparecem em muitas situações.

Exemplos:

- contar quantos alunos foram aprovados;
- contar quantos produtos foram cadastrados;
- contar quantas tentativas de senha foram feitas;
- contar quantos números pares foram digitados;
- controlar a quantidade de repetições em um `while`.

O contador responde perguntas como:

Exemplo de contador com while

```
contador = 1

while contador <= 5:
    print(f"Execução número {contador}")
    contador = contador + 1

print("Fim")
```

Neste exemplo:

- `contador = 1` inicia o controle;
- `contador <= 5` define até quando repete;
- `contador = contador + 1` atualiza o contador.

Sem a atualização, o laço poderia ficar infinito.

Atividade 2 — Teste de mesa com contador

Sem executar no computador, faça o teste de mesa do código.

```
contador = 1

while contador <= 4:
    print(contador)
    contador = contador + 1

print("Terminou")
```

Monte uma tabela:

Rodada	valor de contador	condição `contador <= 4`	saída
1			

Rodada	valor de contador	condição `contador <= 4`	saída
2			
3			
4			

Acumulador

Um **acumulador** é uma variável usada para somar valores ao longo do programa.

Exemplo:

```
total = 0

total = total + 10
total = total + 25
total = total + 15

print(total)
```

Resultado:

```
50
```

O acumulador guarda o resultado parcial da soma.

Contador e acumulador são iguais?

Não.

O **contador** normalmente aumenta em uma quantidade fixa.

```
quantidade = quantidade + 1
```

O **acumulador** soma valores que podem variar.

```
total = total + valor
```

Exemplo:

- contador: quantidade de produtos vendidos;
- acumulador: valor total das vendas.

Exemplo: somando vendas

```
quantidade_vendas = int(input("Quantas vendas deseja registrar? "))

total_vendas = 0

for contador in range(quantidade_vendas):
    valor = float(input("Digite o valor da venda: "))
    total_vendas = total_vendas + valor

print(f"Total vendido: R$ {total_vendas:.2f}")
```

Observe:

- `quantidade_vendas` define quantas vezes o `for` repete;
- `valor` recebe uma venda por vez;
- `total_vendas` acumula todos os valores.

Atividade 3 — Total de vendas

Crie um arquivo chamado:

`vendas_total.py`

O programa deve:

1. perguntar quantas vendas serão registradas;
2. receber o valor de cada venda;
3. calcular o total vendido;
4. mostrar o total ao final.

Teste com:

```
Venda 1: 100
Venda 2: 50
```

Calculando média com acumulador

Para calcular uma média, precisamos de duas informações:

```
soma dos valores
quantidade de valores
```

Fórmula:

```
media = soma / quantidade
```

Em programação:

- usamos um acumulador para somar;
- usamos uma quantidade fixa ou um contador para dividir.

Exemplo: média da turma

```python
quantidade_alunos = int(input("Quantos alunos existem na turma? "))

soma_notas = 0

for contador in range(quantidade_alunos):
    nota = float(input("Digite a nota do aluno: "))
    soma_notas = soma_notas + nota

media_turma = soma_notas / quantidade_alunos

print(f"Média da turma: {media_turma:.2f}")
```

Esse programa funciona, mas ainda tem um problema:

> E se o usuário digitar quantidade zero?

Validação de dados

Validar dados significa verificar se a informação digitada pelo usuário faz sentido.

Exemplos:

- idade não pode ser negativa;
- nota deve estar entre 0 e 10;
- quantidade de alunos deve ser maior que zero;
- valor de compra não deve ser negativo;
- opção de menu deve existir.

A validação evita erros e melhora a qualidade do programa.

Exemplo de validação com while

```
quantidade_alunos = int(input("Quantos alunos existem na turma? "))

while quantidade_alunos <= 0:
    print("A quantidade precisa ser maior que zero.")
    quantidade_alunos = int(input("Digite novamente: "))

print("Quantidade válida.")
```

O `while` repete enquanto o dado estiver incorreto.

Quando o usuário digita um valor válido, o programa continua.

Atividade 4 — Validando quantidade

Crie um arquivo chamado:

`validar_quantidade.py`

O programa deve:

1. solicitar a quantidade de produtos;
2. não aceitar quantidade menor ou igual a zero;
3. repetir a pergunta até o usuário digitar um valor válido;
4. mostrar a quantidade final aceita.

Teste com:

```
0
-3
```

5

O programa deve aceitar apenas o 5.

Validando nota

Agora vamos validar uma nota.

Regra:

```
A nota deve estar entre 0 e 10.
```

Código:

```python
nota = float(input("Digite uma nota entre 0 e 10: "))

while nota < 0 or nota > 10:
    print("Nota inválida.")
    nota = float(input("Digite novamente: "))

print(f"Nota registrada: {nota}")
```

A condição usa `or`, porque basta uma das situações ser verdadeira para a nota ser inválida.

Atividade 5 — Validação de nota

Crie um arquivo chamado:

`validar_nota.py`

O programa deve:

1. pedir uma nota;
2. aceitar apenas valores entre 0 e 10;
3. repetir enquanto o valor for inválido;
4. ao final, mostrar a nota aceita.

Depois responda no caderno:

Média da turma com validação

Agora vamos juntar as ideias.

O programa deve validar:

- quantidade de alunos maior que zero;
- nota entre 0 e 10.

```
quantidade_alunos = int(input("Quantos alunos existem na turma? "))

while quantidade_alunos <= 0:
    print("Quantidade inválida.")
    quantidade_alunos = int(input("Digite novamente: "))

soma_notas = 0

for contador in range(quantidade_alunos):
    nota = float(input("Digite a nota: "))

    while nota < 0 or nota > 10:
```

```
        print("Nota inválida.")
        nota = float(input("Digite novamente: "))

    soma_notas = soma_notas + nota

media = soma_notas / quantidade_alunos
```

Laço aninhado

Um **laço aninhado** é uma repetição dentro de outra repetição.

No exemplo anterior:

```
for contador in range(quantidade_alunos):
    nota = float(input("Digite a nota: "))

    while nota < 0 or nota > 10:
        print("Nota inválida.")
        nota = float(input("Digite novamente: "))
```

Temos:

- um `for` para percorrer os alunos;
- um `while` para validar a nota de cada aluno.

Atividade 6 — Média com validação

Crie um arquivo chamado:

`media_turma_validada.py`

O programa deve:

1. pedir a quantidade de alunos;
2. validar a quantidade;
3. pedir a nota de cada aluno;
4. validar cada nota;
5. calcular a média da turma;
6. mostrar a média final.

Teste digitando uma nota inválida para verificar se a validação funciona.

Contando aprovados e reprovados

Agora vamos gerar um pequeno relatório.

Regra:

```
Média maior ou igual a 7: aprovado
Média menor que 7: reprovado
```

Para isso, podemos usar dois contadores:

```
aprovados = 0
reprovados = 0
```

Durante a repetição, cada contador será atualizado conforme a situação do aluno.

Exemplo: relatório da turma

```
quantidade_alunos = int(input("Quantidade de alunos: "))

aprovados = 0
reprovados = 0
soma_medias = 0

for contador in range(quantidade_alunos):
    media = float(input("Digite a média do aluno: "))
    soma_medias = soma_medias + media

    if media >= 7:
        aprovados = aprovados + 1
    else:
        reprovados = reprovados + 1

media_turma = soma_medias / quantidade_alunos

print(f"Média da turma: {media_turma:.2f}")
```

Atividade 7 — Relatório de notas

Crie um arquivo chamado:

`relatorio_notas.py`

O programa deve:

1. pedir a quantidade de alunos;
2. receber a média de cada aluno;
3. contar aprovados;
4. contar reprovados;
5. calcular a média geral da turma;
6. mostrar um relatório final.

Depois teste com pelo menos cinco alunos.

Repetição com menu e validação

Menus aparecem em muitos sistemas.

Exemplo:

```
1 - Cadastrar venda
2 - Mostrar total
0 - Sair
```

O programa deve continuar rodando enquanto o usuário não escolher sair.

Esse tipo de problema combina:

- `while`;
- `if`, `elif`, `else`;
- contador;
- acumulador;
- validação de opção.

Exemplo: menu de vendas

```
opcao = -1
total = 0
quantidade = 0

while opcao != 0:
    print("1 - Registrar venda")
    print("2 - Mostrar resumo")
    print("0 - Sair")

    opcao = int(input("Escolha uma opção: "))

    if opcao == 1:
        valor = float(input("Valor da venda: "))
        total = total + valor
        quantidade = quantidade + 1
    elif opcao == 2:
        print(f"Quantidade de vendas: {quantidade}")
        print(f"Total vendido: R$ {total:.2f}")
```

Atividade 8 — Menu de caixa

Crie um arquivo chamado:

```
menu_caixa.py
```

O programa deve ter as opções:

```
1 - Registrar venda
2 - Mostrar total vendido
3 - Mostrar quantidade de vendas
0 - Sair
```

Regras:

Erros comuns nesta aula

Alguns erros comuns:

1. esquecer de iniciar o contador ou acumulador;
2. esquecer de atualizar o contador;
3. usar `=` no lugar de `==` em uma condição;
4. validar apenas uma vez em vez de repetir a validação;
5. dividir por zero;
6. deixar a repetição infinita;
7. confundir contador com acumulador.

Esses erros são normais no início.

O importante é testar e corrigir.

Onde estamos no curso

Já estudamos:

- lógica e algoritmos;
- variáveis;
- entrada e saída;
- operadores;
- condições;
- `while`;
- `for`;
- contadores;
- acumuladores;
- validação.

Desafio final da aula

Crie um programa chamado:

`controle_turma.py`

O programa deve:

1. pedir a quantidade de alunos;
2. validar a quantidade;
3. receber nome e média de cada aluno;
4. validar a média;
5. contar aprovados e reprovados;
6. calcular a média geral da turma;
7. mostrar um relatório final.

Exemplo de saída:

```
Quantidade de alunos: 5
Média geral: 7.20
Aprovados: 3
Reprovados: 2
```

Fechamento

Nesta aula, usamos repetições para resolver problemas mais completos.

Trabalhamos com:

- contadores;
- acumuladores;
- validação;
- laços aninhados;
- relatórios simples;
- menus com repetição.

A próxima etapa será aprender a guardar vários dados para consultar e manipular depois.